Sender — armado del hardware (versión 26/09/2026)

Este documento reemplaza el mapa de pines y la parte de alimentación de `TELEMETRY PIMP v6.md` y de `docs/HARDWARE.md`. Lo que no se menciona acá (filtros de las reglas, optoacoplador de la rueda, botón RUN, tira de LEDs) queda igual que antes.

Firmware que corresponde a este armado: `firmware_nuevo/sender_firmware` (ver `CAMBIOS_2026-09-26.md` en esa carpeta).

> **Atención (26/09, mañana; actualizado 07/10):** se aprobaron mejoras que cambian parte de este armado y que **todavía no están en el firmware**: IMU nuevo por SPI en lugar del MPU-6050, BMP580 en lugar del BMP280, medidor de batería MAX17048, PPS del GPS al pin 23, protección de los conectores, botones sin LED y presión de aire de horquilla y shock (2 × ADS1220). Ver `docs/PENDIENTES.md`, mejoras 12–20. El mapa de pines de la sección 2 es el del armado actual; el de la **sección 2.1** es el de la placa definitiva con esas mejoras.

1. Qué cambia respecto del armado anterior

#	Cambio	Por qué
1	**BNO055 y BMP280 pasan al segundo bus I2C: pin 25 (SDA2) y pin 24 (SCL2).** El MPU-6050 queda solo en 18/19.	El MPU se lee 1000 veces por segundo; compartiendo bus, el BNO055 (que estira el reloj del bus) lo podía trabar.
2	**Botón Power: pulsador SIN retención**, entre el pin **On/Off** del Teensy y GND. Reemplaza al pulsador *push-push* que cortaba los 3,7 V.	Permite el auto-apagado y que el firmware cierre la bajada antes de apagarse.
3	**Pila CR1220 en VBAT** del Teensy (+ a VBAT, − a GND).	El reloj sigue en hora con el Sender apagado: las bajadas siempre tienen fecha.
4	**Llave de periféricos (pin 4):** P-MOSFET de nivel lógico + N-MOSFET, corta la batería hacia el step-up de 12 V (MT3608).	Con el Sender apagado no queda nada consumiendo (antes el MT3608 estaba siempre conectado a la batería).
5	**Llave del WiFi (pin 9):** mismo circuito, corta la alimentación del módulo WiFi/BLE.	"Consumo cero" real del WiFi en pista (hueco 7 del README, antes sólo se podía dormir por comando).
6	**Se saca el IRF9530.**	El 12 V ahora sólo existe cuando la llave de periféricos está prendida. Además el IRF9530 no es de nivel lógico (ver punto 4.4).
7	**Módulo Bluetooth Adafruit Bluefruit LE: se saca** cuando llegue el ESP32-C3 (hace WiFi y Bluetooth). Pines 0/1 quedan libres.	Un módulo menos, y los comandos del Bluefruit nunca se pudieron confirmar.
8	**WiFi: ESP32-C3 SuperMini** en lugar del ESP-01S, en el mismo puerto serie (Serial5, pines 20/21).	Mucho más rápido para bajar las bajadas; ver sección 5.

---

2. Mapa de pines del Teensy 4.1

Negrita = cambió o es nuevo.

Pin	Función	Conecta a	Notas
VIN	Alimentación	Batería (salida OUT+ del TP4056 HW-373), **directo**	Rango del Teensy: 3,6–5,5 V. Ver 4.1
On/Off	**Botón Power**	**Pulsador sin retención a GND**	Toque: prende. Mantener 4 s: apagado forzado (ver 4.2)
VBAT	**Pila del reloj**	**CR1220 (+)**; (−) a GND	Portapila para CR1220
3.3V	3,3 V para sensores	MPU-6050, ADXL375, BNO055, BMP280, GPS, reglas KTC, pull-up del PC817	Máximo 250 mA en total. Ver 4.5
GND	Masa común	Todo	
0 (RX1) / 1 (TX1)	~~Bluefruit LE~~	**Libres** cuando se saque el Bluefruit	Mientras siga el Bluefruit, igual que antes
2	Rueda (interrupción)	PC817 pin 4 (colector)	Sin cambios
3	Botón RUN	Pulsador a GND	Sin cambios
4	**Llave de periféricos**	**Gate del N-MOSFET Q2** (vía 1 kΩ)	HIGH = 12 V prendido. Ver 4.3
5	LED del botón RUN	vía 220 Ω	Sin cambios
6	Tira Neopixel	DIN de la tira	Sin cambios
7 (RX2) / 8 (TX2)	GPS NEO-F10N	TX / RX del GPS (cruzados)	El firmware lo pasa a 115200 baudios y 10 Hz solo
9	**Llave del WiFi**	**Gate del N-MOSFET Q4** (vía 1 kΩ)	HIGH = WiFi prendido (sólo en boxes)
10 / 11 / 12 / 13	SPI: CS / MOSI / MISO / SCK	ADXL375	Sin cambios
16 (A2)	Batería	Punto medio del divisor 10k/10k	Sin cambios
18 (SDA) / 19 (SCL)	**I2C rápido**	**Sólo el MPU-6050**	Antes compartido con BNO055 y BMP280
20 (TX5) / 21 (RX5)	WiFi	ESP-01S hoy / **ESP32-C3 después** (ver 5)	Cruzados: TX5 → RX del módulo
24 (SCL2) / 25 (SDA2)	**I2C lento**	**BNO055 y BMP280**	Nuevo bus
36 (A12)	Horquilla KTC-200	Filtro VK200 + 100 Ω + 10 µF	Conector 1. Sin cambios
37 (A13)	Shock KTC-100	Mismo filtro	Conector 2. Sin cambios
38 (A14) / 39 (A15)	Expansión	Conectores 4 y 5	Sin cambios

Pines libres para el futuro: 0, 1 (cuando se saque el Bluefruit), 14, 15, 17, 22, 23, 26–33, 34, 35, 40, 41.

2.1 Mapa de pines de la placa definitiva (mejoras 12–20, aprobado sin armar)

Sólo lo que cambia respecto de la tabla de arriba. El resto queda igual.

Pin	Función	Conecta a	Mejora
0 / 1	Libres	— (se saca el Bluefruit)	—
5	Libre	— (el botón RUN ya no lleva LED)	19
10–13	SPI compartido	ADXL375 (CS 10) + IMU ICM-42688-P (CS 26) + 2 × ADS1220 (CS 27 y 28)	17, 20
14 / 15	Digital conectores 1 / 2	DS18B20 (1-Wire), pull-up 4,7 kΩ + 100 Ω en serie	9, 12

Pin	Función	Conecta a	Mejora
16	Libre	— (la batería la mide el MAX17048)	14
18 / 19	Libres	— (se saca el MPU-6050)	17
22	Botón Power (lectura)	Diodo Schottky al botón	Decisión abierta 1
23	PPS del GPS	Salida PPS del NEO-F10N	15
24 / 25	I2C lento	BNO055 + BMP580 + MAX17048	14, 18
26	CS del IMU	ICM-42688-P	17
27	**CS presión horquilla**	**ADS1220 #1**, pull-up 10 kΩ a 3,3 V	**20**
28	**CS presión shock**	**ADS1220 #2**, pull-up 10 kΩ a 3,3 V	**20**
29	**DRDY presión horquilla**	**ADS1220 #1**	**20**
30	**DRDY presión shock**	**ADS1220 #2**	**20**
40 / 41	Digital conectores 4 / 5	Switch de freno u otro (pull-up 4,7 kΩ)	9

Conectores 1 y 2 (horquilla y shock): **M8 de 8 polos** — 1 3,3 V (después del PTC), 2 GND, 3 regla, 4 DS18B20, 5 presión S+, 6 presión S−, 7 reserva, 8 malla. Conectores 4 y 5: universales de 4 polos. Detalle del circuito de presión en `docs/HARDWARE.md`, sección 7.

Pines libres en la placa definitiva: 0, 1, 5, 16, 17, 18, 19, 31–35.

---

3. Buses de sensores

3.1 I2C rápido (Wire, pines 18/19) — 400 kHz

- **Sólo el MPU-6050** (dirección 0x68).
- Pull-ups: los que trae la plaquita GY-521 (4,7 kΩ). No agregar más.
- Cables lo más cortos posible.

3.2 I2C lento (Wire2, pines 25 SDA2 / 24 SCL2) — 400 kHz

- **BNO055** (Adafruit, dirección 0x28).
- **BMP280** (Adafruit, dirección **0x77** — la plaquita de Adafruit trae SDO a 3,3 V. El firmware prueba 0x77 y después 0x76, así que cualquiera sirve).
- Pull-ups: las dos plaquitas de Adafruit traen 10 kΩ cada una; en paralelo quedan 5 kΩ, está bien. No agregar más.

3.3 SPI (pines 10–13) — ADXL375

Sin cambios. El firmware ahora verifica que responda (DEVID = 0xE5) y lo configura a 1600 muestras por segundo.

3.4 Vigilante del I2C (firmware)

Si un sensor I2C se cuelga, el firmware no se traba: descarta esa lectura, libera el bus (pulsos de reloj + STOP, ~150 µs), deja descansar ese bus 5 ms y sigue grabando a 1000 Hz. Cada falla queda anotada en la bajada. No hace falta nada de hardware para esto.

Nota: el límite de 5 µs que planteamos no es posible: leer los 14 bytes del MPU-6050 tarda unos 400 µs de por sí. El presupuesto quedó en 700 µs para el bus rápido y 1,5 ms para el lento.

---

4. Alimentación

4.1 Esquema general

```
                        ┌──────────────────────────────── VIN  Teensy 4.1
LiPo ── TP4056 ─────────┤                                  (se prende/apaga con On/Off)
3,7 V   HW-373 OUT+     │
                        ├── Llave periféricos (pin 4) ── MT3608 ── +12V ── anillos LED
                        │   Q1 P-MOSFET / Q2 N-MOSFET     (step-up)       └─ LED sensor rueda
                        │
                        └── Llave WiFi (pin 9) ──────────── regulador 3,3 V ── módulo WiFi/BLE
                            Q3 P-MOSFET / Q4 N-MOSFET     (buck-boost)

Teensy 3.3V ── MPU-6050, ADXL375, BNO055, BMP280, GPS, reglas KTC, pull-up PC817
Teensy VBAT ── CR1220
Teensy On/Off ── pulsador Power ── GND
```

4.2 Botón Power (pin On/Off del Teensy)

- **Pulsador SIN retención** (momentáneo) entre **On/Off** y **GND**. El pulsador metálico M16 con anillo LED viene en versión momentánea: pedir esa. El anillo LED se alimenta del +12 V de la llave de periféricos, así que se prende cuando el Sender está prendido.
- **Toque corto:** prende el Sender.
- **Apagar:** lo normal es dejar que se apague solo (4.6). Si hace falta apagarlo a mano: **primero parar la grabación con RUN** y después mantener Power 4 segundos. Ese apagado es por hardware y no cierra el archivo (igual que el botón anterior): si se apaga grabando, se pierde como mucho el último segundo de la bajada.
- **En evaluación** (ver `PENDIENTES.md`): conectar el botón también al pin 22 con un diodo, para que "mantener 2 s" haga un apagado prolijo por firmware.

4.3 Llave de periféricos (pin 4) y llave del WiFi (pin 9)

Las dos son el mismo circuito:

```
BAT (+3,7…4,2 V) ──────┬──────────────────── S ┐
                       │                      Q1 / Q3  P-MOSFET de nivel lógico
                    100 kΩ                    │         (AO3401A, SI2301, o NDP6020P
                       │                      │          si lo querés en TO-220)
                       ├──────────────────── G ┘
                       │                      D ──────────── salida (BAT_SW / BAT_WIFI)
                    1 kΩ
                       │
                       D ┐
                         Q2 / Q4  N-MOSFET (2N7000, BS170 o 2N7002)
Pin 4 / Pin 9 ─ 1 kΩ ─ G ┤
                    100 kΩ   S ┘
                       │     │
GND ───────────────────┴─────┴──
```

- Pin del Teensy en HIGH → Q2 conduce → gate de Q1 a GND → Q1 conduce → hay tensión en la salida.
- Pin en LOW (o Teensy apagado) → el 100 kΩ lleva el gate de Q1 a la batería → Q1 cortado → salida en 0 V.
- El 100 kΩ entre gate y GND de Q2 mantiene todo apagado mientras el Teensy arranca.

4.4 Por qué no el IRF9530

El IRF9530 necesita unos −10 V entre gate y source para conducir bien. Con una batería de 3,7 V como mucho llega a −4 V: quedaría a medio conducir y se calentaría con carga. Además, con 3,3 V en el gate no se lo puede cortar si su source está en la batería. Donde estaba (cortando 12 V) funcionaba, pero con este esquema ya no hace falta.

4.5 Consumos sobre el 3,3 V del Teensy (máximo recomendado: 250 mA)

Carga	Consumo aproximado
GPS NEO-F10N (doble banda)	~30–40 mA (verificar en la hoja de datos)
BNO055	~12 mA
MPU-6050	~4 mA
BMP280, ADXL375	< 1 mA c/u
Reglas KTC (según su resistencia)	~1 mA c/u
Total	**~60 mA**

El módulo WiFi/BLE **no** va al 3,3 V del Teensy: pide picos de 300–400 mA. Va con su propio regulador después de la llave del pin 9.

Tira Neopixel: no está documentado de dónde se alimenta. **No** tiene que ir al 3,3 V del Teensy: cada LED puede pedir hasta 60 mA. Conviene alimentarla de la llave de periféricos (BAT_SW, 3,7–4,2 V) y así también se apaga con el Sender.

4.6 Auto-apagado (firmware)

Si la bici está **quieta** (según el acelerómetro del MPU-6050) y **nadie tocó RUN, ni pasó la rueda, ni hubo pedidos desde la app por WiFi durante 10 minutos**, el firmware:

1. si estaba grabando, anota el evento y **cierra la bajada de forma segura**;
2. apaga el WiFi y la llave de periféricos;
3. se apaga a sí mismo por software (mismo mecanismo que el pin On/Off).

Se vuelve a prender con un toque del botón Power. El reloj sigue en hora con la CR1220. Los 10 minutos se cambian en `config.h` (`MINUTOS_AUTO_APAGADO`).

4.7 Watchdog (firmware)

Si el programa se trabara más de 2 segundos por cualquier motivo (SD, un sensor, un error), el procesador se reinicia solo. La bajada en curso queda guardada hasta el último bloque escrito.

4.8 USB

El puente VIN–VUSB del Teensy **ya está cortado**: se puede enchufar el USB para programar con la batería conectada sin problema.

---

5. Módulo WiFi/BLE

Hoy: ESP-01S

- Serial5: pin 20 (TX5) → RX del ESP-01S, pin 21 (RX5) ← TX del ESP-01S.
- Alimentación: salida de la llave del pin 9 → regulador 3,3 V → VCC y CH_PD del ESP-01S (CH_PD puenteado a su 3,3 V, como antes).

Próximo: ESP32-C3 SuperMini

- Serial5: pin 20 (TX5) → **GPIO20** (RX) del C3; pin 21 (RX5) ← **GPIO21** (TX) del C3.
- Alimentación: salida de la llave del pin 9 → **regulador 3,3 V buck-boost** → pin **3V3** del C3 (no al pin 5V).
- El C3 reemplaza también al Bluefruit LE (pulsómetro/potencia por Bluetooth).
- Probar el alcance del WiFi apenas llegue: hay tandas de SuperMini con la antena mal ubicada. Si el alcance es corto, se corrige bajando la potencia de transmisión en su firmware.
- Su firmware y el cambio en el Teensy son el próximo paso de software.

---

6. Primera prueba en banco (en este orden)

1. **Alimentación:** con la batería conectada y el Sender apagado, medir que no haya tensión en la salida de las dos llaves. Tocar Power: el Teensy prende, el pin 4 pasa a HIGH y aparece el 12 V; la salida de la llave del WiFi sigue en 0 V.
2. **Monitor serie (USB, 115200):** el firmware avisa al arrancar si algún sensor no responde:
   - `ADXL375 no responde (DEVID != 0xE5)`
   - `MPU-6050 no responde en el bus rapido (18/19)`
   - `BNO055 no responde en el bus lento (24/25)`
   - `BMP280 no responde en 0x77 ni 0x76 (bus lento 24/25)`
3. **GPS:** en exterior, confirmar que el LED de GPS pase a fix. Grabar una bajada corta y verla en la app: la traza del mapa tiene que tener ~10 puntos por segundo. Si no, revisar la configuración UBX con u-center.
4. **Grabación:** grabar 3 minutos y abrir la bajada en la app. Revisar los avisos de la bajada: no debería haber bloques descartados ni fallas de I2C.
5. **Reloj:** mandar la config desde la app (pone la hora), apagar el Sender, esperar unos minutos, prender y grabar: el archivo tiene que salir con la fecha y hora correctas.
6. **WiFi:** mantener RUN 4 s: se tiene que prender el módulo (pin 9 en HIGH) y aparecer la red. Arrancar a grabar: se tiene que cortar.
7. **Auto-apagado:** para no esperar 10 minutos, poner temporalmente `MINUTOS_AUTO_APAGADO = 1` en `config.h`, dejar la bici quieta y confirmar que se apaga solo y que la bajada quedó cerrada.
8. **Watchdog:** agregar temporalmente `while (1) {}` al final del `loop()` y confirmar que el Teensy se reinicia solo a los ~2 segundos. Sacarlo después.

---

7. Lista de compras para estos cambios

> La lista completa y al día (con todos los componentes del Sender, estado y precio) está en `docs/Sender_componentes.xlsx`. Esta tabla queda como resumen de lo que agregan los cambios del 26/09.

Cantidad	Componente	Para
1	Pulsador metálico M16 **momentáneo** con anillo LED 12 V	Botón Power
1	Pila CR1220 + portapila	Reloj
2	P-MOSFET de nivel lógico (AO3401A / SI2301 SMD, o NDP6020P TO-220)	Q1, Q3
2	N-MOSFET 2N7000 / BS170 (TO-92) o 2N7002 (SMD)	Q2, Q4
4	Resistencia 100 kΩ	Gates
4	Resistencia 1 kΩ	Gates
1	Regulador 3,3 V buck-boost, ≥ 500 mA (p. ej. módulo TPS63020 o Pololu S7V8F3)	Módulo WiFi/BLE
1–2	ESP32-C3 SuperMini	WiFi + Bluetooth